

metafrontier: Unified Metafrontier Analysis for Efficiency and Productivity in R

by Erik Enstad

Abstract Comparing the technical efficiency of groups of firms that face different restrictions of a common production technology requires a metafrontier framework that envelops the group-specific frontiers. Existing R packages cover parts of this workflow, but none combines SFA- and DEA-based metafrontier estimation with inference for technology gap ratios and the metafrontier Malmquist decomposition. We introduce metafrontier, a package that provides a unified interface for three complementary approaches: the deterministic metafrontier of Battese, Rao, and O'Donnell (2004) under both of its identification criteria, the stochastic metafrontier of Huang, Huang, and Liu (2014), and DEA-based metafrontier models with radial, directional, hyperbolic, and FDH technologies. The package estimates group-specific frontiers, constructs the metafrontier envelope, and decomposes efficiency into group technical efficiency and the technology gap ratio. Additional features include the metafrontier Malmquist productivity index with three-way decomposition, bootstrap confidence intervals for technology gap ratios, Murphy–Topel corrected standard errors, poolability tests for both SFA and DEA fits, convergence diagnostics, latent class metafrontier estimation via the EM algorithm, panel stochastic frontier models with time-varying inefficiency on balanced or unbalanced panels, and ggplot2 visualisation methods. Pre-fitted group frontiers from sfaR and frontier are accepted directly, and estimation can be delegated to sfaR, frontier, or Benchmarking as backends. We illustrate the package with simulated and real data and a Monte Carlo study covering parameter and efficiency recovery, small samples, and unbalanced group sizes. All examples and simulations are fully reproducible and complete in under ten minutes on a standard laptop.

1 Introduction

Firms within the same industry often face different restrictions on the set of production plans available to them. In microeconomic terms the industry technology set contains all technically feasible input-output combinations, but regulation, environmental conditions, resource endowments, and the cost of switching production techniques can prevent particular groups of firms from accessing every point in it. Steel producers, for example, operate either the capital-intensive blast furnace–basic oxygen furnace (BF-BOF) route or the scrap-based electric arc furnace (EAF) route; both belong to the same industry technology set, but capital vintage and scrap availability restrict which points an individual producer can reach. Similarly, dairy farms in different countries face distinct climatic conditions and input markets that restrict their feasible production plans (Moreira and Bravo-Ureta, 2010). When groups of producers face different restricted technology sets, pooling all observations into a single stochastic frontier or data envelopment analysis model conflates within-group inefficiency with between-group differences in the accessible technology, potentially yielding misleading efficiency rankings.

The metafrontier framework, introduced by Battese et al. (2004) and extended by O'Donnell et al. (2008) and Huang et al. (2014), addresses this problem by defining an overarching production frontier, the *metafrontier*, that envelops the group-specific frontiers. This framework decomposes a firm's distance from the metafrontier into two components:

$$TE_i^* = TE_i \times TGR_i \quad (1)$$

where TE_i is the firm's *technical efficiency* relative to its own group frontier, measuring managerial performance within the available technology, and TGR_i is the *technology gap ratio*, measuring how close the group's frontier lies to the best-practice metafrontier. The product TE_i^* is the firm's *metafrontier technical efficiency*, that is, its performance measured against the industry-wide best practice. Figure 1 in Section 2.2 illustrates this decomposition for the single-input case.

The package is designed for applied economists, regulators, and operations researchers who need to compare technical efficiency across groups operating under different production technologies.

Several R (R Core Team, 2025) packages address parts of the efficiency analysis workflow. The *frontier* package (Coelli and Henningsen, 2020), the *sfaR* package (Dakpo et al., 2023), and the newer *sfa* package (Bernstein et al., 2026) provide stochastic frontier estimation across a wide set of specifications but include no metafrontier layer. The *Benchmarking* package (Bogetoft and Otto, 2011) implements a wide range of DEA models; its XREF/YREF arguments score observations against an

externally supplied reference technology, so pooled-frontier DEA scores can be assembled manually, but the user must orchestrate the per-group runs, form and summarise the technology gap ratios, and forgo inference on them: its bootstrap targets Farrell scores with respect to a single technology, and its Malmquist functions compute the standard index rather than the metafrontier decomposition. The **deaR** package (Coll-Serrano et al., 2023) offers a general DEA toolkit, including directional distance functions via `model_dir()`, but no metafrontier workflow. The **smfa** package (Owili, 2026), the CRAN release of the earlier GitHub package **metafrontierR** (Olieko, 2026), wraps **sfaR** to estimate deterministic (LP and QP), stochastic, latent class, and sample selection (Greene, 2010) metafrontiers, but it is SFA-only: it provides no DEA-based metafrontier, no productivity decomposition, no bootstrap or Murphy–Topel inference for technology gap ratios, and no panel or visualisation support. No package, on CRAN or elsewhere, combines SFA- and DEA-based metafrontier estimation with inference for technology gap ratios and the metafrontier Malmquist decomposition of O'Donnell et al. (2008).

The **metafrontier** package fills this gap. Its principal contributions are:

- **Unified estimation framework.** A single function, `metafrontier()`, dispatches across SFA and DEA methods with deterministic or stochastic metafrontier formulations through the `method` and `meta_type` arguments.
- **Malmquist productivity decomposition.** The `malmquist_meta()` function computes the metafrontier Malmquist total factor productivity index with the three-way decomposition of O'Donnell et al. (2008).
- **Bootstrap confidence intervals.** The `boot_tgr()` function provides parametric and nonparametric bootstrap inference for technology gap ratios, with parallel computation support.
- **Murphy–Topel corrected standard errors.** The stochastic metafrontier accounts for the generated-regressor problem using the variance correction of Murphy and Topel (1985).
- **Latent class metafrontier.** The `latent_class_metafrontier()` function estimates group membership endogenously when technology classes are unobserved.
- **Interoperability.** Pre-fitted group frontiers from **sfaR** and **frontier** are accepted directly via the `models` argument, and the engine argument delegates group-frontier estimation to **sfaR**, **frontier**, or (for DEA) **Benchmarking**.
- **Diagnostics and user safety.** Every estimation stage records its convergence status, inspectable via `check_convergence()` and `summary()`; poolability of the group frontiers is testable for both SFA (likelihood ratio) and DEA (permutation) fits.

The remainder of this article is organised as follows. Section 2.2 reviews the metafrontier methodology. Section 2.3 describes the package design. Section 2.4 demonstrates basic usage with simulated data. Section 2.5 covers advanced features. Section 2.6 presents Monte Carlo evidence on estimator performance. Section 2.7 compares **metafrontier** with related packages, and Section 2.8 concludes.

2 Metafrontier methodology

Readers already familiar with stochastic frontier analysis and metafrontier models may wish to proceed directly to Section 2.3.

Throughout, we use the term *technology* in the sense of the metafrontier literature. The industry *metatechnology* T^* is the technology set of microeconomic production theory: the set of all input-output combinations that are technically feasible given the current state of knowledge in the industry (O'Donnell et al., 2008). Group-specific *technologies* $T_j \subset T^*$ are restricted subsets of the metatechnology, arising because regulation, environmental conditions, resource endowments, or the cost of switching production techniques prevent the firms in group j from choosing every point of T^* . Following the convention of Battese et al. (2004) and O'Donnell et al. (2008), we refer to these restricted sets simply as group technologies; the metafrontier is the boundary of T^* , and the group frontiers are the boundaries of the sets T_j .

2.1 Group-specific stochastic frontiers

Consider J groups of firms, where group j ($j = 1, \dots, J$) has access to a potentially restricted technology set T_j . The stochastic frontier production function for group j is

$$y_{ij} = \mathbf{x}_{ij}'\boldsymbol{\beta}_j + v_{ij} - u_{ij}, \quad i = 1, \dots, N_j \quad (2)$$

where y_{ij} is the log output of firm i in group j , $\mathbf{x}_{ij}$ is a vector of log inputs (including a constant), $\boldsymbol{\beta}_j$ is the group-specific parameter vector, $v_{ij} \sim N(0, \sigma_{v,j}^2)$ is a symmetric noise term, and $u_{ij} \geq 0$ is a one-sided inefficiency term (Aigner et al., 1977). The composed error is $\varepsilon_{ij} = v_{ij} - u_{ij}$.

The **metafrontier** package supports three distributional assumptions for u_{ij} : the half-normal ($u_{ij} \sim |N(0, \sigma_{u,j}^2)|$), the truncated normal ($u_{ij} \sim N^+(\mu_j, \sigma_{u,j}^2)$), and the exponential ($u_{ij} \sim \text{Exp}(\lambda_j)$). Two estimators of technical efficiency from the composed error are in common use, and the package computes both. The default is the conditional expectation of Battese and Coelli (1988) (BC88),

$$\text{TE}_{ij} = E[\exp(-u_{ij}) | \varepsilon_{ij}] = \frac{\Phi(\mu_{ij}^*/\sigma_j^* - \sigma_j^*)}{\Phi(\mu_{ij}^*/\sigma_j^*)} \exp(-\mu_{ij}^* + \frac{1}{2}\sigma_j^{*2}) \quad (3)$$

where μ_{ij}^* and σ_j^* are the mean and standard deviation of the conditional distribution of u_{ij} given ε_{ij} , and $\Phi(\cdot)$ is the standard normal distribution function. BC88 provides consistent estimates of technical efficiency scores. The alternative estimator $\exp(-\hat{u}_{ij})$ with $\hat{u}_{ij} = E[u_{ij} | \varepsilon_{ij}]$ (Jondrow et al., 1982, JLMS) does not account for Jensen's inequality; the difference is negligible when σ_u is small relative to σ_v but grows for highly inefficient samples. The estimator argument of `metafrontier()` selects between the two ("bc88", the default, or "jlms"); both are stored on the fitted object, and `efficiencies()` can switch between them without refitting. For a book-length treatment of stochastic frontier analysis, see Kumbhakar and Lovell (2000).

2.2 Deterministic metafrontier

Battese et al. (2004; building on Battese and Rao, 2002) define the metafrontier as the deterministic envelope of the group-specific frontiers. The metafrontier production function satisfies $f^*(\mathbf{x}) \geq f_j(\mathbf{x})$ for all $\mathbf{x}$ and all j , meaning that it represents the output that would be attainable if all firms had access to the most productive technology available in the industry.

Let $\hat{\beta}_j$ denote the maximum likelihood estimate from group j , and let $\hat{f}_{ij} = \mathbf{x}'_{ij}\hat{\beta}_j$ denote the fitted group frontier value for observation i in group j . The **metafrontier** package obtains β^* by solving the linear program

$$\min_{\beta^*} \sum_{i=1}^N \mathbf{x}'_i \beta^* \quad \text{s.t.} \quad \mathbf{x}'_{ij} \beta^* \geq \hat{f}_{ij} \quad \forall i, j \quad (4)$$

which minimises the total metafrontier output subject to the envelope constraint, producing the tightest feasible metafrontier. Battese et al. (2004) propose two criteria for identifying β^* under the same envelope constraints: minimising the sum of absolute deviations, which reduces to the linear program in Equation (4) because the constraints force every deviation to be non-negative (O'Donnell et al., 2008, Eqs. 23–25), and minimising the sum of squared deviations, a convex quadratic program. The two share the same constraint set but differ in objective and can yield different β^* when not all constraints are binding; Section 2.6 reports their maximum absolute TGR difference on the simulation designs. The package implements both: `objective = "lp"` (the default) is solved via **IpSolveAPI** (Konis and Schwendinger, 2024), and `objective = "qp"` is solved exactly via **quadprog** (Turlach and Weingessel, 2019), with an adaptive-barrier fallback via `constrOptim()` when **quadprog** is unavailable. The technology gap ratio under the deterministic metafrontier is

$$\text{TGR}_{ij} = \frac{\exp(\mathbf{x}'_{ij}\hat{\beta}_j)}{\exp(\mathbf{x}'_{ij}\hat{\beta}^*)} \quad (5)$$

which lies in $[0, 1]$ by construction. Because the metafrontier is deterministic, no standard errors are available for β^* without resampling methods.

2.3 Stochastic metafrontier

Huang et al. (2014) propose an alternative in which the metafrontier itself is stochastic. Define the fitted group frontier value $\hat{y}_{ij} = \mathbf{x}'_{ij}\hat{\beta}_j$. The stochastic metafrontier is estimated as a second-stage SFA:

$$\hat{y}_{ij} = \mathbf{x}'_{ij}\beta^* + v_{ij}^* - u_{ij}^* \quad (6)$$

where v_{ij}^* captures approximation noise and $u_{ij}^* \geq 0$ represents the technology gap. This formulation provides standard errors and hypothesis tests for β^* , overcoming a key limitation of the deterministic approach. Note that because the stochastic metafrontier need not envelop the group frontiers at every point (due to the noise term v_{ij}^*), the resulting TGR can exceed 1 for some observations: at those points the group frontier locally surpasses the estimated metafrontier. How often this occurs is

sample-dependent: in the Monte Carlo design used below no observation reaches $TGR > 1$, whereas on the `sfaR` utility example used later around 40% of observations do, with values up to about 1.25. Such excess values reflect noise rather than superior-to-best-practice technology. For group-level means or ranking applications we recommend leaving them untruncated so the noise averages out; for firm-level interpretation we recommend truncating at one ($\text{pmin}(tgr, 1)$) and reporting the truncated share as a diagnostic.

However, because the dependent variable $\hat{y}_{ij}$ is itself estimated from the first stage, the standard errors of β^* are understated unless corrected for the generated-regressor problem. The `metafrontier` package addresses this with the variance correction of [Murphy and Topel \(1985\)](#), as described in Section 2.5.

2.4 DEA-based metafrontier

The metafrontier framework extends naturally to nonparametric methods ([O'Donnell et al., 2008](#); [Charnes et al., 1978](#)). For each group j , a data envelopment analysis (DEA) frontier is estimated using only the observations in that group, yielding group-specific efficiency scores TE_{ij}^{group} . A second DEA problem pools all observations across groups, producing metafrontier scores TE_{ij}^{pool} . In the output orientation (the default), the score of observation (x_i, y_i) against a reference set R is $TE_i = 1/\phi_i^*$, where ϕ_i^* solves the envelopment linear program

$$\max_{\phi_i, \lambda} \phi_i \quad \text{s.t.} \quad \sum_{j \in R} \lambda_j x_j \leq x_i, \quad \sum_{j \in R} \lambda_j y_j \geq \phi_i y_i, \quad \lambda \in \Lambda, \quad (7)$$

and in the input orientation the score is θ_i^* from $\min \theta_i$ subject to $\sum_j \lambda_j x_j \leq \theta_i x_i$, $\sum_j \lambda_j y_j \geq y_i$, $\lambda \in \Lambda$. The set Λ encodes the returns to scale: $\lambda_j \geq 0$ unrestricted (CRS), adding $\sum_j \lambda_j = 1$ (VRS), $\sum_j \lambda_j \leq 1$ (DRS), $\sum_j \lambda_j \geq 1$ (IRS), or $\lambda_j \in \{0, 1\}$ with $\sum_j \lambda_j = 1$ (FDH, dropping convexity). The group-frontier problem restricts the reference set R to the observations of group j ; the metafrontier problem pools all N observations into R . This distinction is the entire metafrontier construction: the same LP is solved twice per observation against nested reference sets. The technology gap ratio is then

$$TGR_{ij} = \frac{TE_{ij}^{\text{pool}}}{TE_{ij}^{\text{group}}} \quad (8)$$

which lies in $(0, 1]$ because the pooled reference set contains the group reference set. Beyond radial measures, the DEA path exposes directional distance functions (DDF) via `type = "directional"`, with direction either a preset ("proportional", "output", "input"), a common numeric direction vector, or a matrix of firm-specific directions, and hyperbolic (graph) efficiency via `type = "hyperbolic"`, which contracts inputs and expands outputs simultaneously and remains feasible in cross-period comparisons where radial cross-period programs may not be. Second-stage slack maximisation is available via `slack = TRUE`, flagging observations that are radially efficient but not Koopmans-efficient. The resulting object carries the same metafrontier class and supports the standard extractor and plotting methods. Because DEA is nonparametric, it avoids functional form assumptions, but it does not account for statistical noise: positive noise realisations shift the estimated frontier upwards, while negative realisations are attributed to inefficiency. SFA, by contrast, accounts for noise through the composed error, although neither approach can separate the unobserved noise and inefficiency realisations for an individual observation; SFA efficiency scores are conditional expectations (Section 2.2.1).

2.5 Efficiency decomposition

Regardless of the estimation method, the fundamental identity holds:

$$TE_{ij}^* = TE_{ij} \times TGR_{ij} \quad (9)$$

where TE_{ij} measures how efficiently firm i uses its own group's technology, TGR_{ij} measures how close that group's technology is to the best-practice metafrontier, and TE_{ij}^* measures overall performance relative to the industry-wide frontier.

Figure 1 illustrates this decomposition for a single-input production function. The vertical distance from a firm's observed output to its group frontier measures technical inefficiency, while the distance from the group frontier to the metafrontier measures the technology gap. A firm can be technically efficient within its group ($TE_i = 1$) yet still fall short of the metafrontier if its group's technology is

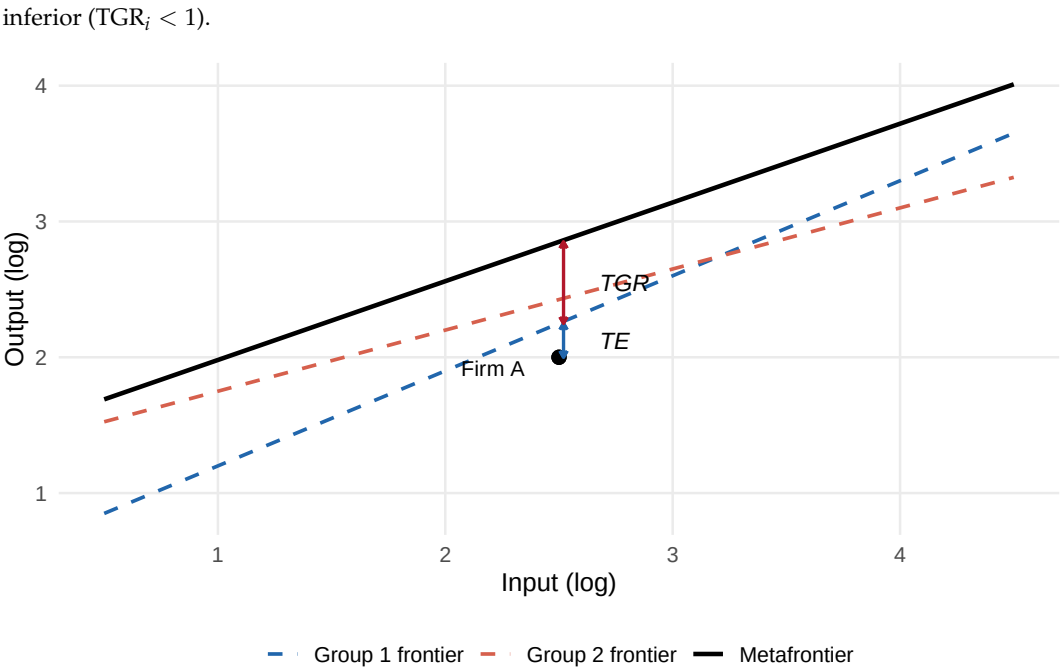


Figure 1: Metafrontier decomposition for a single-input production function (diagram drawn with ggplot2; code in the supplementary materials). For Firm A, technical efficiency (TE) is the distance from observed output to the group frontier, the technology gap ratio (TGR) the distance from the group frontier to the metafrontier (solid black), and $TE^* = TE \times TGR$.

Table 1 summarises the key properties of the three metafrontier approaches.

Table 1: Comparison of metafrontier approaches.

Property	Deterministic MF	Stochastic MF	DEA
Functional form	Parametric	Parametric	Nonparametric
Statistical noise	First stage only	Both stages	None
Inference for metafrontier	None (resample)	Yes (with M–T correction)	None (bootstrap)
TGR bounds	[0, 1]	Can exceed 1	[0, 1]
Key reference	BRO (2004)	HHL (2014)	ORB (2008)

2.6 Malmquist productivity decomposition

For panel data, the metafrontier Malmquist total factor productivity (TFP) index extends the standard Malmquist index to incorporate technology gap dynamics (O'Donnell et al., 2008). The index between periods t and $t + 1$ decomposes as

$$M_{i,t,t+1}^* = TEC_{i,t,t+1} \times TGC_{i,t,t+1} \times TC_{i,t,t+1}^*$$

(10)

where TEC is the *technical efficiency change* (catching up to the group frontier), TGC is the *technology gap change* (movement of the group frontier relative to the metafrontier), and TC^* is the *metafrontier technical change* (shift in the metafrontier itself). This three-way decomposition allows researchers to distinguish productivity growth driven by improved managerial practice from gains attributable to technology adoption or frontier innovation.

One interpretive caveat applies. The Malmquist index is not a *proper* productivity index in the sense of O'Donnell (2012): except under restrictive conditions on the technology, it is not multiplicatively complete and fails the transitivity axiom of index theory, so chained multi-period or cross-firm comparisons of index levels are not generally valid (O'Donnell, 2018). Interpretation should be confined to binary period-to-period comparisons, which is how the package reports the index. Multiplicatively complete alternatives such as the Färe-Primont and Lowe indexes are planned extensions (Section 2.8).

3 Package design and architecture

3.1 Design principles and S3 interface

The `metafrontier` package is designed around two principles: a unified interface and a consistent S3 method suite. The package exports 13 standalone functions and registers 38 S3 methods across five object classes: `metafrontier` (the primary estimation result), `malmquist_meta` (productivity decomposition), `boot_tgr` (bootstrap inference), `lc_metafrontier` (latent class models), and `metafrontier_convergence` (diagnostics). Table 2 lists all 16 user-facing functions grouped by category, including 3 autoplot methods that are registered as S3 methods of the `ggplot2` generic rather than standalone exports.

Table 2: User-facing functions in the metafrontier package.

Category	Function	Description
Estimation	<code>'metafrontier()'</code>	Main metafrontier estimation (SFA or DEA)
Estimation	<code>'malmquist_meta()'</code>	Metafrontier Malmquist TFP index
Estimation	<code>'latent_class_metafrontier()'</code>	EM algorithm for latent class metafrontier
Analysis	<code>'efficiencies()'</code>	Extract TE, TGR, or TE* scores
Analysis	<code>'technology_gap_ratio()'</code>	Technology gap ratios by group
Analysis	<code>'tgr_summary()'</code>	Summary statistics for TGR
Analysis	<code>'poolability_test()'</code>	Poolability test (LR for SFA, permutation for DEA)
Analysis	<code>'check_convergence()'</code>	Convergence diagnostics for all estimation stages
Inference	<code>'boot_tgr()'</code>	Bootstrap CIs for technology gap ratios
Inference	<code>'select_n_classes()'</code>	BIC-based selection of latent classes
Inference	<code>'as_metafrontier_model()'</code>	Convert external models to metafrontier format
Visualisation	<code>'autoplot.metafrontier()'</code>	TGR density, efficiency, decomposition, frontier plots
Visualisation	<code>'autoplot.malmquist_meta()'</code>	Malmquist decomposition, TGR evolution, MPI trend plots
Visualisation	<code>'autoplot.boot_tgr()'</code>	Bootstrap distribution and CI plots
Simulation	<code>'simulate_metafrontier()'</code>	Cross-sectional simulation with known DGP
Simulation	<code>'simulate_panel_metafrontier()'</code>	Panel simulation with time-varying inefficiency

A single `metafrontier()` entry point was chosen over separate functions for each method–meta-type combination because users frequently compare estimators on the same data; a unified interface guarantees that the returned objects share the same S3 class and methods regardless of the estimation path. S3 was preferred over S4 and R6 because it provides the lightest-weight dispatch mechanism, requires no additional dependencies, and matches the class system used by the upstream packages (`frontier`, `Benchmarking`) with which `metafrontier` interoperates.

All primary estimation is accessed through `metafrontier()`, which dispatches via its method argument (`"sfa"` or `"dea"`) and `meta_type` argument (`"deterministic"` or `"stochastic"`). The returned `metafrontier` object supports the standard R model interface: `summary()`, `coef()`, `vcov()`, `confint()`, `fitted()`, `residuals()`, `logLik()`, `nobs()`, `predict()`, and `plot()`, so fitted objects work with the standard model-handling idiom without adaptation.

3.2 Estimation workflow

The package supports three paths to metafrontier estimation. In the **formula path**, the user supplies raw data, a formula, and a group identifier; the package estimates all group-specific frontiers internally before constructing the metafrontier. In the **pre-fitted model path**, the user first estimates group frontiers using external packages (`sfaR` or `frontier`) and passes the fitted objects directly to `metafrontier()` via the `models` argument; conversion to the internal format happens automatically. In the **engine path**, the user stays in the formula interface but delegates the group-frontier estimation to an external backend via the `engine` argument: `engine = "sfaR"` or `engine = "frontier"` for SFA group frontiers, and `engine = "Benchmarking"` for the DEA path, where both the group frontiers and the pooled metafrontier are computed by `Benchmarking::dea()` using its XREF/YREF external-reference facility.

The formula path is the simplest workflow. A single call to `metafrontier()` handles all estimation steps: parsing the formula, splitting data by group, estimating group-specific frontiers, constructing the metafrontier envelope, and computing the TGR decomposition. The pre-fitted model and engine paths are useful when the user requires fine-grained control over individual group frontier specifications or needs features of the specialised packages that the internal estimators lack. The metafrontier stage of the SFA methods is always estimated internally, because the Murphy–Topel variance correction (Section 2.5) differentiates the internal stage-2 likelihood.

3.3 Interoperability

Fitted model objects from [sfaR](#) (class `sfacross`) and [frontier](#) (class `frontier`) can be passed directly to the `models` argument of `metafrontier()`; the conversion to the internal format is performed automatically by the `as_metafrontier_model()` generic, which users only need to call explicitly for hand-built list models (the conversion is idempotent, so pre-converting is harmless). Farrell objects from [Benchmarking](#) store neither coefficients nor the underlying data, so they cannot be used with models; the `engine = "Benchmarking"` path covers that use case instead. The following example illustrates the pre-fitted model path (not evaluated):

```
library(sfaR)
# Estimate group frontiers externally
mod1 <- sfacross(log_y ~ log_x1 + log_x2,
  data = subset(dat, group == "G1"))
mod2 <- sfacross(log_y ~ log_x1 + log_x2,
  data = subset(dat, group == "G2"))

# Pass the fitted objects directly; conversion is automatic
fit <- metafrontier(models = list(G1 = mod1, G2 = mod2))
```

3.4 Computational details

The package implements its own maximum likelihood estimator for group-specific stochastic frontiers, using BFGS optimisation with Nelder–Mead fallback for robustness. Variance parameters are log-transformed to ensure unconstrained optimisation over the positive real line. The deterministic metafrontier of Equation (4) is solved via linear programming using the [lpSolveAPI](#) package ([Konis and Schwendinger, 2024](#)), with LP object reuse across observations for improved performance on large datasets. The squared-deviations variant (`objective = "qp"`) is solved exactly by the dual method of Goldfarb and Idnani as implemented in [quadprog](#) ([Turlach and Weingessel, 2019](#)), which is listed in `Suggests` to keep the import footprint unchanged. Numerical stability is maintained through a safe Mills ratio computation (`.safe_mills()`) that prevents NaN values in extreme tail regions of the normal distribution, relevant for both panel SFA and latent class models.

The package keeps its dependency footprint minimal. The seven Imports (`stats`, `graphics`, `grDevices`, and methods from base R; `Formula` for multi-part formula parsing; `numDeriv` for Hessian computation; and `lpSolveAPI` for LP solving) are each exercised by core functionality. [lpSolveAPI](#) was chosen over alternatives such as [Rglpk](#) because it is a self-contained C library with no external system dependencies, so it installs without additional system libraries on all supported platforms. Packages used only in optional features ([ggplot2](#) for `autoplot()` methods, [parallel](#) for bootstrap parallelisation, [quadprog](#) for the QP objective, and the three upstream SFA/DEA engines for interoperability and backend delegation) are listed as `Suggests` and loaded conditionally.

Correctness is supported by a test suite of 22 [testthat](#) files (over 125 test blocks) covering core SFA/DEA estimation, Monte Carlo recovery, bootstrap, Murphy–Topel correction, panel SFA, latent class, DDF, and [ggplot2](#) `autoplot` methods, plus snapshot tests of S3 outputs. Continuous integration runs the suite on Ubuntu, Windows, and macOS under R-devel, release, and oldrel. The custom MLE is additionally benchmarked against [frontier](#) (`test-frontier-equivalence.R`): on a 3-group dataset from `simulate_metafrontier()` with 150 firms per group and technology gaps of 0, 0.3, and 0.5 (seed 42), similar to the simulation introduced in Section 2.4.1, the maximum absolute coefficient difference against `frontier::sfa()` is 1.2×10^{-5} for group 2 and 5.0×10^{-6} for group 3. The two estimators use different internal parameterisations but deliver numerically equivalent slope and intercept estimates.

Computational environment. All timings reported in this article were obtained on a laptop with an Intel Core Ultra 5 125U processor and 32 GB of RAM running Windows 11 x64, using R version 4.5.0 (2025-04-11) and [metafrontier](#) 0.3.1. The complete article code, including the Monte Carlo study of Section 2.6, runs in under ten minutes in this environment.

4 Basic usage

All examples in this section use simulated data for full reproducibility. The [metafrontier](#) package includes data-generating functions that produce data with known true parameters, so estimator accuracy can be assessed directly.

4.1 Simulating metafrontier data

The `simulate_metafrontier()` function creates cross-sectional data with `n_groups` technology groups, each containing `n_per_group` firms. The data-generating process uses a Cobb–Douglas production function with group-specific intercepts that induce known technology gaps.

```
sim <- simulate_metafrontier(n_groups = 3, n_per_group = 200, seed = 42)
dat <- sim$data
head(dat[, c("log_y", "log_x1", "log_x2", "group",
             "true_te", "true_tgr")])
#>      log_y  log_x1  log_x2 group  true_te true_tgr
#> 1 3.371473 4.574030 4.4255884   G1 0.6700025      1
#> 2 3.665773 4.685377 2.5855553   G1 0.7704492      1
#> 3 2.784899 1.430698 4.2596549   G1 0.9834917      1
#> 4 3.916103 4.152238 2.2139813   G1 0.9853877      1
#> 5 2.313365 3.208728 0.7894005   G1 0.8407115      1
#> 6 2.210272 2.595480 2.2116232   G1 0.7410986      1
```

The returned list contains `$data` (the simulated data frame) and `$params` (the true parameter values). The data frame includes the true technical efficiency (`true_te`), true technology gap ratio (`true_tgr`), and true metafrontier efficiency (`true_te_star`), so estimates can be compared directly with estimated values.

4.2 Deterministic metafrontier

The deterministic metafrontier of Battese et al. (2004) is estimated by specifying `meta_type = "deterministic"`:

```
fit_det <- metafrontier(log_y ~ log_x1 + log_x2, data = dat,
                       group = "group", method = "sfa",
                       meta_type = "deterministic", dist = "hnormal")
summary(fit_det)
#>
#> Metafrontier Model Summary
#> =====
#>
#> Call:
#> metafrontier(formula = log_y ~ log_x1 + log_x2, data = dat, group = "group",
#>   method = "sfa", meta_type = "deterministic", dist = "hnormal")
#>
#> Method:      sfa
#> Metafrontier: deterministic
#>
#> --- Group: G1 (n = 200) ---
#>           Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  1.04540    0.07899  13.234 < 2e-16 ***
#> log_x1        0.49023    0.01351  36.295 < 2e-16 ***
#> log_x2        0.19249    0.01378  13.972 < 2e-16 ***
#> log_sigma_v  -1.51266    0.16168  -9.356 < 2e-16 ***
#> log_sigma_u  -1.18597    0.24943  -4.755 1.99e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> Log-likelihood: -33.087
#>
#> --- Group: G2 (n = 200) ---
#>           Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.77078    0.10811   7.130 1.01e-12 ***
#> log_x1        0.47875    0.01450  33.022 < 2e-16 ***
#> log_x2        0.19243    0.01413  13.623 < 2e-16 ***
#> log_sigma_v  -1.38894    0.16414  -8.462 < 2e-16 ***
#> log_sigma_u  -1.53414    0.59564  -2.576    0.01 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> Log-likelihood: -29.889
```

```

#>
#> --- Group: G3 (n = 200) ---
#>               Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.55554    0.06185   8.982  <2e-16 ***
#> log_x1       0.50554    0.01265  39.956  <2e-16 ***
#> log_x2       0.18212    0.01221  14.916  <2e-16 ***
#> log_sigma_v -1.77273    0.14227 -12.460  <2e-16 ***
#> log_sigma_u -1.05299    0.12265  -8.585  <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> Log-likelihood: -18.492
#>
#> --- Metafrontier ---
#>               Estimate
#> (Intercept)  1.0454
#> log_x1       0.4902
#> log_x2       0.1925
#>
#> --- Efficiency Decomposition ---
#>   Group Mean_TE Mean_TGR Mean_TE_star
#>   G1   0.7962   1.0000   0.7962
#>   G2   0.8488   0.7385   0.6269
#>   G3   0.7739   0.6223   0.4816
#>
#> --- Technology Gap Ratio Summary ---
#>   Group  N  Mean    SD   Min    Q1 Median    Q3   Max
#>   G1  200 1.0000 0.0000 1.0000 1.0000 1.0000 1.0000 1.0000
#>   G2  200 0.7385 0.0118 0.7178 0.7284 0.7380 0.7481 0.7598
#>   G3  200 0.6223 0.0169 0.5838 0.6111 0.6218 0.6346 0.6580
#>
#> --- Convergence ---
#> All estimation stages converged.

```

The `summary()` method reports the group-specific frontier coefficients, the metafrontier coefficients obtained from the constrained optimisation of Equation (4), and summary statistics for the technology gap ratios by group.

4.3 Stochastic metafrontier

The stochastic metafrontier of [Huang et al. \(2014\)](#) replaces the deterministic envelope with a second-stage SFA:

```

fit_sto <- metafrontier(log_y ~ log_x1 + log_x2, data = dat,
                        group = "group", method = "sfa",
                        meta_type = "stochastic", dist = "hnormal")

coef(fit_sto)
#> (Intercept)      log_x1      log_x2
#>  0.7975464    0.4928374    0.1867363
confint(fit_sto)
#>           2.5 %    97.5 %
#> (Intercept) 0.5578845 1.0372084
#> log_x1      0.4817555 0.5039193
#> log_x2      0.1758149 0.1976577

```

Unlike the deterministic approach, the stochastic metafrontier provides standard errors, z-statistics, and confidence intervals for the metafrontier coefficients directly.

4.4 DEA-based metafrontier

The DEA-based metafrontier computes group-specific and pooled DEA frontiers without parametric assumptions. Because DEA requires strictly positive inputs and outputs, we use level-scale variables rather than the log-transformed variables used for SFA:

```

dat_lev <- within(dat, {
  y <- exp(log_y)
  x1 <- exp(log_x1)
  x2 <- exp(log_x2)
})
fit_dea <- metafrontier(y ~ x1 + x2, data = dat_lev,
  group = "group", method = "dea", rts = "vrs")
tgr_summary(fit_dea)
#>   Group   N      Mean      SD      Min      Q1      Median
#> 1    G1  200 0.9760509 0.05474776 0.5343945 0.9755096 1.0000000
#> 2    G2  200 0.8426304 0.09659937 0.6513259 0.7672661 0.8480490
#> 3    G3  200 0.5905357 0.11012920 0.4058857 0.5332346 0.5655475
#>           Q3 Max
#> 1 1.0000000    1
#> 2 0.9155486    1
#> 3 0.6110953    1

```

4.5 Extracting results and diagnostics

The `efficiencies()` function extracts efficiency scores. Three types are available: group technical efficiency ("group"), metafrontier efficiency ("meta"), and technology gap ratios ("tgr"). The fundamental identity of Equation (9) can be verified numerically:

```

te <- efficiencies(fit_det, type = "group")
tgr <- efficiencies(fit_det, type = "tgr")
te_star <- efficiencies(fit_det, type = "meta")
all.equal(te_star, te * tgr, tolerance = 1e-10)
#> [1] TRUE

```

The `tgr_summary()` function provides descriptive statistics for the TGR by group:

```

tgr_summary(fit_det)
#>   Group   N      Mean      SD      Min      Q1      Median
#> 1    G1  200 1.0000000 1.084493e-13 1.0000000 1.0000000 1.0000000
#> 2    G2  200 0.7385079 1.184913e-02 0.7177953 0.7284223 0.7379621
#> 3    G3  200 0.6223039 1.691828e-02 0.5838489 0.6110806 0.6218278
#>           Q3      Max
#> 1 1.0000000 1.0000000
#> 2 0.7481009 0.7597913
#> 3 0.6345533 0.6579921

```

The `poolability_test()` function tests the null hypothesis that all groups share a common frontier against the alternative of group-specific frontiers. For SFA fits it conducts a likelihood ratio test; for DEA fits, where no likelihood exists, it dispatches to a permutation test that exploits the exchangeability of group labels under the pooled-technology null (Simar and Zelenyuk, 2007; Kneip et al., 2016):

```

poolability_test(fit_det)
#>
#> Likelihood Ratio Test for Poolability of Group Frontiers
#>
#> data: fit_det
#> LR = 275.4, df = 10, p-value < 2.2e-16

```

Convergence of every estimation stage can be verified with `check_convergence()`, which reports one row per group frontier plus the metafrontier stage. The same information is printed by `summary()`, and a warning is issued at fit time whenever an optimiser stops at a non-zero code. Estimation aborts with an error only when no usable estimate exists; otherwise the codes are recorded so that users can inspect them before interpreting TGRs, confidence intervals, or productivity decompositions:

```

check_convergence(fit_det)
#>
#> Convergence of estimation stages
#> -----

```

```
#>      stage method code converged note
#> group: G1     MLE    0      TRUE
#> group: G2     MLE    0      TRUE
#> group: G3     MLE    0      TRUE
#> metafrontier  LP     0      TRUE
```

The `ggplot2` (Wickham, 2016)-based `autoplot()` method provides four plot types. Figure 2 shows the distribution of technology gap ratios by group, and Figure 3 displays the efficiency decomposition.

```
autoplot(fit_det, which = "tgr")
```

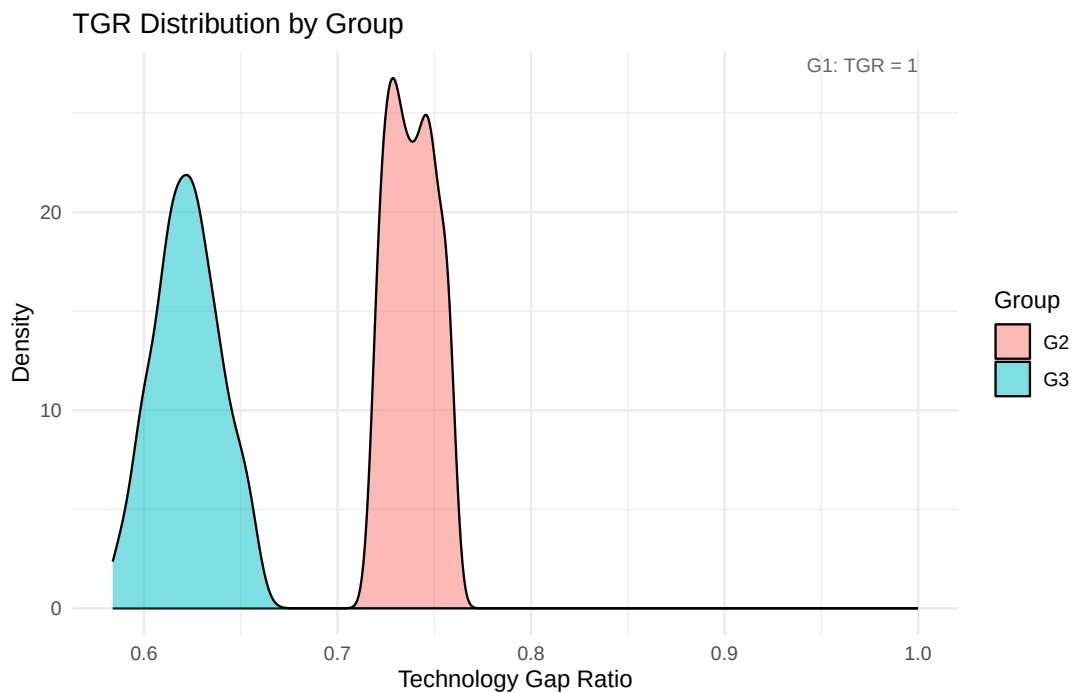


Figure 2: Distribution of technology gap ratios by group from the deterministic metafrontier (generated with the package `autoplot()` method, view: TGR density). Group G1 defines the metafrontier (TGR = 1.000 for all observations, annotated separately); groups G2 and G3 show increasing technology gaps.

```
autoplot(fit_det, which = "decomposition")
```

4.6 Real-data illustration

The examples above use simulated data with known parameters. To demonstrate the package on empirical data, we use the utility dataset from `sfaR`, which contains 791 observations on US fossil-fuel steam electric generating plants over 1986–1996. The binary variable `regu` equals one for plants in states that enacted retail-access (restructuring) legislation during the sample period and zero for plants that remained under traditional regulation. This gives a natural grouping of plants. The production function relates output (y) to capital (k), labour (labor), and fuel (fuel).

```
data("utility", package = "sfaR")
utility$group <- ifelse(utility$regu == 1, "Retail access", "Regulated")
utility$log_y <- log(utility$y)
utility$log_k <- log(utility$k)
utility$log_labor <- log(utility$labor)
utility$log_fuel <- log(utility$fuel)

fit_util <- metafrontier(log_y ~ log_k + log_labor + log_fuel,
  data = utility, group = "group",
  method = "sfa", meta_type = "deterministic")
```

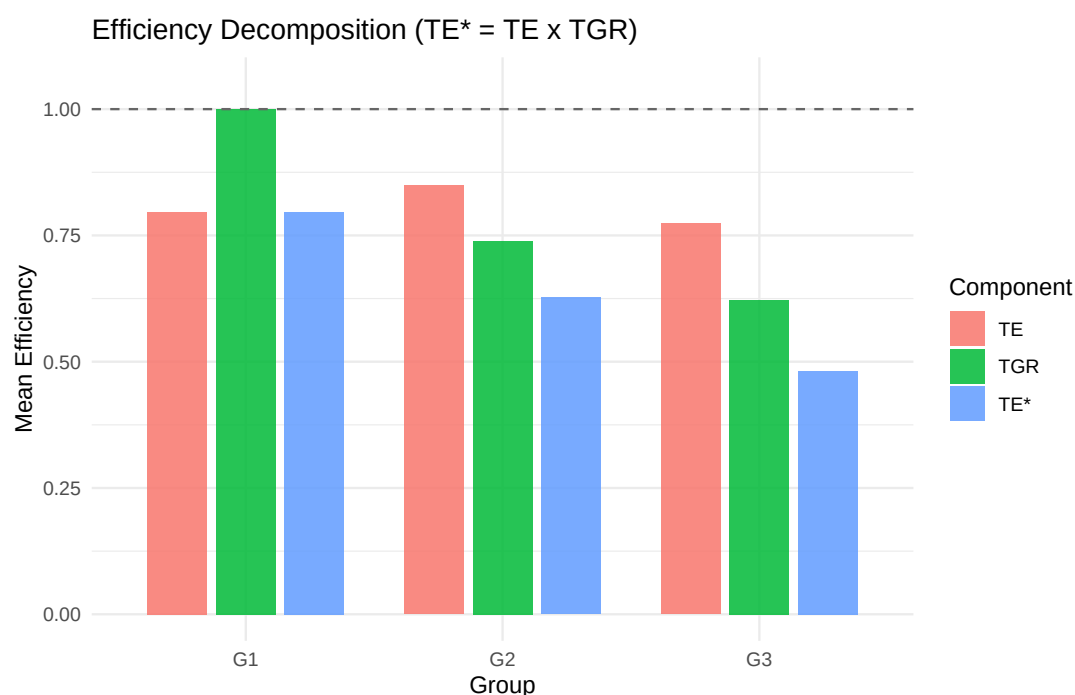


Figure 3: Mean efficiency decomposition by group (generated with the package `autoplot()` method, view: decomposition). The three bars per group show mean technical efficiency (TE), technology gap ratio (TGR), and metafrontier efficiency $TE^* = TE \times TGR$.

```
tgr_summary(fit_util)
#>      Group   N      Mean      SD      Min      Q1      Median
#> 1   Regulated 297 0.9457963 0.02197346 0.8755055 0.9318836 0.9423030
#> 2 Retail access 494 0.8920070 0.05876240 0.6279105 0.8614020 0.8973676
#>      Q3 Max
#> 1 0.9584353 1
#> 2 0.9327646 1
```

The traditionally regulated group exhibits a slightly higher mean TGR (0.946) than the group of plants in retail-access states (0.892); the regulated plants lie closer to the pooled metafrontier on average in this sample. We attach no structural interpretation to this illustrative comparison. In the metafrontier framework, a technology gap reflects restrictions on the feasible input-output set; regulatory constraints that only affect the choice of input mix would manifest as allocative rather than technical inefficiency and would not, by themselves, generate a technology gap.

Because all three estimators share the same interface and object class, cross-method robustness checks require no additional plumbing. The following chunk re-estimates the same model with the stochastic metafrontier and with DEA, then compares group-mean TGRs and the Spearman rank correlations of the plant-level TGRs across estimators:

```
fit_util_st <- metafrontier(log_y ~ log_k + log_labor + log_fuel,
                           data = utility, group = "group",
                           method = "sfa", meta_type = "stochastic")
fit_util_dea <- metafrontier(log_y ~ log_k + log_labor + log_fuel,
                             data = utility, group = "group",
                             method = "dea", rts = "vrs")

tgr_by <- function(fit) tapply(efficiencies(fit, type = "tgr"),
                               utility$group, mean)
round(rbind(Deterministic = tgr_by(fit_util),
           Stochastic     = tgr_by(fit_util_st),
           DEA             = tgr_by(fit_util_dea)), 3)
#>      Regulated Retail access
#> Deterministic    0.946      0.892
#> Stochastic      1.041      0.975
#> DEA             0.996      NA
```

```
round(c(det_vs_stoch = cor(fit_util$tgr, fit_util_st$tgr,
                          method = "spearman"),
      det_vs_dea    = cor(fit_util$tgr, fit_util_dea$tgr,
                          method = "spearman")), 3)
#> det_vs_stoch  det_vs_dea
#>      0.858      NA
```

Agreement in the group ranking across a deterministic envelope, a distributional second stage, and a nonparametric pooled frontier indicates that the technology gap is a feature of the data rather than an artefact of any single method; this kind of triangulation is the main practical payoff of a unified interface.

5 Advanced features

5.1 Bootstrap confidence intervals

Because the deterministic metafrontier provides no standard errors for the technology gap ratios, the `boot_tgr()` function implements bootstrap inference. Both parametric (residual resampling) and nonparametric (case resampling) approaches are supported, with percentile and bias-corrected accelerated (BCa) interval types.

```
boot <- boot_tgr(fit_det, R = 199, type = "parametric",
                ci_type = "percentile", seed = 1, progress = FALSE)
boot_ci <- confint(boot)
head(boot_ci, 5)
#>      2.5% 97.5%
#> [1,] 0.9589637 1
#> [2,] 0.9751020 1
#> [3,] 0.9349823 1
#> [4,] 0.9784612 1
#> [5,] 0.9782767 1
```

We use $R = 199$ here to keep the article reproducible in minutes; in applied work, $R \geq 999$ is recommended for reliable percentile intervals, and $R \geq 1999$ for BCa intervals.

The `confint()` method returns observation-level confidence intervals; Figure 4 displays the bootstrap distribution of the group-level means of the TGRs, and Table 3 summarises the intervals by group. Parallel computation is available via the `ncores` argument, which uses the **parallel** package for distributing replications across cores. Table 3 is produced by the following (echoed) chunk.

```
autoplot(boot)

ci_df <- data.frame(
  Group = dat$group,
  Lower = boot_ci[, 1],
  Upper = boot_ci[, 2],
  TGR = efficiencies(fit_det, type = "tgr")
)
ci_summary <- aggregate(cbind(TGR, Lower, Upper) ~ Group,
                        data = ci_df, FUN = mean)
names(ci_summary) <- c("Group", "Mean TGR", "Mean lower bound",
                      "Mean upper bound")
cap <- paste("Bootstrap 95%% confidence intervals for TGR by group",
            "(group-level means of observation-level intervals).")
knitr::kable(ci_summary, caption = cap, digits = 4,
             align = "lccc", booktabs = TRUE, linesep = "") |>
  ktbl()
```

5.2 Murphy–Topel variance correction

The stochastic metafrontier of Huang et al. (2014) uses fitted values from the first-stage group SFA models as the dependent variable in a second-stage SFA. This two-step procedure understates the

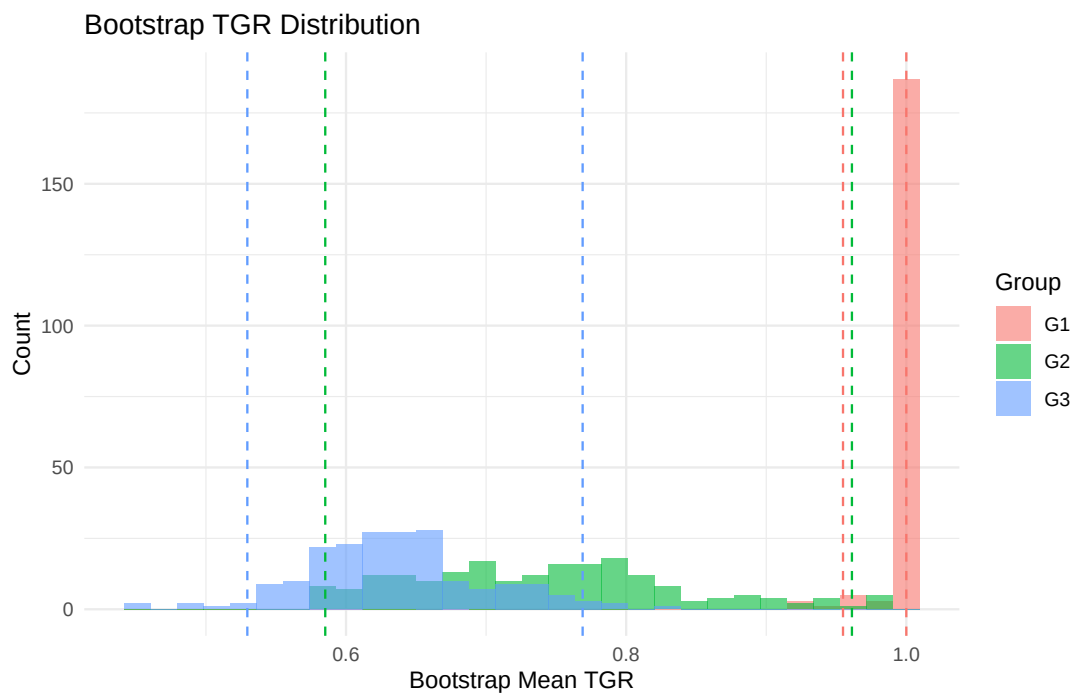


Figure 4: Bootstrap distribution of the group-level mean technology gap ratios, from 199 parametric replications (generated with the package `autoplot()` method for bootstrap objects, default view). Dashed vertical lines mark the 95 percent percentile confidence intervals for the group means.

Table 3: Bootstrap 95\% confidence intervals for TGR by group (group-level means of observation-level intervals).

Group	Mean TGR	Mean lower bound	Mean upper bound
G1	1.0000	0.9584	1.0000
G2	0.7385	0.5828	0.9680
G3	0.6223	0.5195	0.7794

standard errors of the metafrontier coefficients because it ignores the sampling variability from the first stage (the generated-regressor problem). The `vcov()` method implements the correction of [Murphy and Topel \(1985\)](#), which inflates the second-stage variance-covariance matrix to account for first-stage estimation uncertainty.

```
se_uncorrected <- sqrt(diag(vcov(fit_sto)))
se_corrected <- sqrt(diag(vcov(fit_sto, correction = "murphy-topel")))
comparison <- data.frame(
  Uncorrected = se_uncorrected,
  `Murphy-Topel` = se_corrected,
  `Ratio` = se_corrected / se_uncorrected,
  check.names = FALSE
)
knitr::kable(comparison, digits = 4, booktabs = TRUE, linesep = "",
  caption = paste("Comparison of uncorrected and",
    "Murphy--Topel corrected standard errors",
    "for the stochastic metafrontier coefficients.")
) |> ktbl()
```

The corrected standard errors in Table 4 are uniformly larger; the increase is the sampling uncertainty contributed by the generated regressors. Confidence intervals computed with `confint(fit_sto, correction = "murphy-topel")` use the corrected standard errors. The correction is computed via numerical differentiation of the second-stage log-likelihood with respect to the first-stage parameters, requiring $O(p_1 \times p_2)$ likelihood evaluations per group, where p_1 is the number of first-stage parameters of that group's frontier (frontier coefficients plus variance parameters) and p_2 is the number of second-stage metafrontier parameters. For typical metafrontier specifications (3–5 regressors) this com-

Table 4: Comparison of uncorrected and Murphy–Topel corrected standard errors for the stochastic metafrontier coefficients.

	Uncorrected	Murphy-Topel	Ratio
(Intercept)	0.1223	0.1328	1.0861
log_x1	0.0057	0.0095	1.6759
log_x2	0.0056	0.0099	1.7703

pletes in seconds, but the cost scales quadratically and may become noticeable with high-dimensional translog models.

5.3 Heteroscedastic SFA

The package supports heteroscedastic group frontiers through the formula interface. A two-part formula $y \sim x_1 + x_2 \mid z_1$ models the mean of $\sigma_{u,i}$ (for half-normal and exponential) or μ_i (for truncated-normal) as a function of environmental variables:

```
fit_het <- metafrontier(log_y ~ log_x1 + log_x2 | log_x1, data = dat,
                        group = "group", method = "sfa",
                        meta_type = "deterministic", dist = "hnormal")
coef(fit_het)
#> (Intercept)      log_x1      log_x2
#>  1.0465785    0.4897621    0.1924461
```

This specification allows the degree of inefficiency to vary with observable firm characteristics within each group, which is relevant when the variance of inefficiency is not constant across producers. Heteroscedasticity enters only the group TE scores: the metafrontier envelope depends on $\mathbf{x}'\hat{\beta}_i$, not on variance parameters, so TGRs are unaffected while TE and TE* reflect firm-specific inefficiency.

5.4 Latent class metafrontier

When group membership is unobserved, the `latent_class_metafrontier()` function discovers technology classes endogenously and estimates the metafrontier simultaneously using an EM algorithm. Unlike `metafrontier()`, this function does not accept a group argument: class membership is determined entirely by the data. The number of latent classes can be selected using BIC via `select_n_classes()`.

```
lc <- latent_class_metafrontier(log_y ~ log_x1 + log_x2, data = dat,
                               n_classes = 3, n_starts = 5, seed = 42)
coef(lc)
#> (Intercept)      log_x1      log_x2
#>  0.8829379    0.5897582    0.2895326
```

As with all latent class stochastic frontier models, estimation may be sensitive to starting values and local optima. The EM algorithm terminates when the relative change in marginal log-likelihood between iterations falls below a user-controllable tolerance (`tol`, default 10^{-6}) or after `max_iter` iterations (default 200). The package therefore supports multiple random starts via the `n_starts` argument (default 10) and information-criteria-based model selection via `select_n_classes()`, which fits models for each class count in `n_classes_range` and returns the BIC-optimal specification. For applied work we recommend at least `n_starts = 10` for two or three classes, increased to 20–30 for four or more classes or when the likelihood surface is known to be multimodal.

5.5 Panel SFA with time-varying inefficiency

For panel data, the package supports the time-varying inefficiency specifications of Battese and Coelli (1992) (BC92) and Battese and Coelli (1995) (BC95). The BC92 model parameterises inefficiency as $u_{it} = u_i \cdot \exp[-\eta(t - T)]$, where u_i is a firm-specific truncated-normal draw, T is the final period of the panel, $\eta > 0$ implies declining inefficiency over time, and $\eta < 0$ implies increasing inefficiency. Unbalanced panels are supported: each firm contributes its own set of periods $\mathcal{T}_i$ to the likelihood,

and the decay term is still measured from the panel-wide final period T , following the unbalanced formulation of Battese and Coelli (1992).

The BC95 model instead specifies observation-specific inefficiency $u_{it} \sim N^+(z'_{it}\delta, \sigma_u^2)$, where z_{it} is a vector of inefficiency determinants and δ is estimated jointly with the frontier in a single stage by maximum likelihood. Unlike BC92, the BC95 model treats observations as independent draws (no within-firm time dependence). It is selected with `panel_dist = "bc95"` and requires the two-part formula syntax $y \sim x_1 + x_2 \mid z_1 + z_2$ introduced for the heteroscedastic models above.

```
psim <- simulate_panel_metafrontier(
  n_groups = 2, n_firms_per_group = 30, n_periods = 5,
  eta = 0.05, seed = 42
)
pdat <- psim$data

fit_panel <- metafrontier(log_y ~ log_x1 + log_x2, data = pdat,
  group = "group", method = "sfa",
  meta_type = "deterministic",
  panel = list(id = "firm", time = "year"),
  panel_dist = "bc92")

coef(fit_panel)
#> (Intercept)      log_x1      log_x2
#>  1.0308915    0.5246135    0.2988751
tgr_summary(fit_panel)
#>   Group  N      Mean          SD      Min      Q1      Median
#> 1    G1 150 1.0000000 1.439400e-13 1.0000000 1.0000000 1.0000000
#> 2    G2 150 0.6095756 1.122649e-02 0.5705535 0.6009255 0.6098201
#>           Q3      Max
#> 1 1.0000000 1.0000000
#> 2 0.6169438 0.6365206
```

All estimated parameters, including the decay parameter η and the variance parameters, are available with `coef(fit_panel, which = "group", extraPar = TRUE)`, and the group tables printed by `summary()` report η with its standard error.

Panel SFA can be combined with either `meta_type = "deterministic"` (as above) or `meta_type = "stochastic"`; in the latter case the group-specific time-varying fits are fed into the second-stage stochastic metafrontier exactly as in the cross-sectional case.

5.6 Malmquist productivity decomposition

The `malmquist_meta()` function computes the metafrontier Malmquist TFP index with the three-way decomposition of Equation (10). Firms are matched across periods by the identifier supplied through the `id` argument; observations without a match in the adjacent period (unbalanced panels, entry and exit, or firms that switch groups) are dropped from the affected transition with a warning that reports the count. When `id` is omitted the function falls back to matching by row position within each group, which is only valid for balanced panels sorted identically in every period, and says so in a message. Under variable, decreasing, or increasing returns to scale, cross-period DEA programs can be genuinely infeasible when an observation lies outside the comparison technology; such cases yield NA (never infinite values), are excluded from reported means, and are counted in a consolidated warning and in the printed output. Constant returns to scale (used below) avoids the issue. The SFA path (`method = "sfa"`) approximates metafrontier distances by the pointwise maximum over the estimated group frontiers and announces this in a message; the DEA path computes exact distances to the pooled frontier and is the default. The identity $M^* = TEC \times TGC \times TC^*$ can be verified numerically.

```
pdat_lev <- within(pdat, {
  y <- exp(log_y)
  x1 <- exp(log_x1)
  x2 <- exp(log_x2)
})
malm <- malmquist_meta(y ~ x1 + x2, data = pdat_lev,
  group = "group", time = "year", id = "firm",
  method = "dea", rts = "crs")

print(malm)
#>
```

```
#> Metafrontier Malmquist TFP Index
#> =====
#> Method:      dea
#> Orientation: output
#> RTS:         crs
#> Groups:      G1, G2
#> Periods:     1 -> 2 -> 3 -> 4 -> 5
#> Observations: 240
#>
#> Mean decomposition (M* = TEC x TGC x TC*):
#>   MPI  = 1.084
#>   TEC  = 1.029
#>   TGC  = 1.055
#>   TC*  = 1.028
```

Figure 5 displays the Malmquist decomposition over time. Each point is plotted at the end period of its transition, so the first point, at period 2, is the change from period 1 to period 2.

```
autoplot(malm, which = "mpi_trend")
```

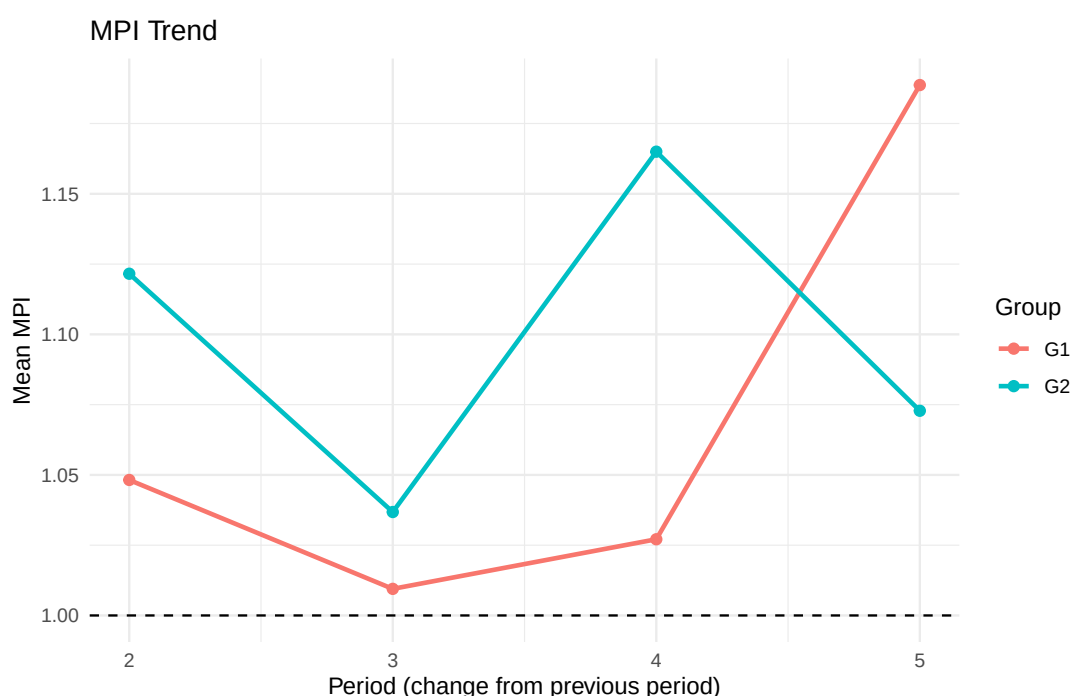


Figure 5: Mean metafrontier Malmquist productivity index (MPI) by period and group (generated with the package `autoplot()` method, view: MPI trend). Each point sits at the end period of its transition; values above the dashed line indicate improvement relative to the previous period.

6 Monte Carlo evidence

A simulation study evaluates the finite-sample performance of the deterministic, stochastic, and DEA metafrontier estimators.

6.1 Design

We study two data-generating processes. **Scenario A** follows the group SFA specification of Equation (2) with two groups ($J = 2$), 100 firms per group, and known true parameters: $\beta^* = (1.0, 0.5, 0.3)'$ (intercept, β_1, β_2), technology gap $\delta = (0, 0.4)$, inefficiency standard deviations $\sigma_u = (0.2, 0.3)$, and noise standard deviation $\sigma_v = 0.15$. Log inputs are drawn independently and uniformly on $[0, 5]$, identically across groups, and the groups differ only in their frontier intercepts, so a common log-linear

metafrontier exists and coefficient recovery is well defined. **Scenario B** relaxes the features of Scenario A that are unrealistic in applications: the three groups have group-specific *slope* coefficients (not merely intercept shifts), log inputs are correlated within firm (correlation 0.6, a firm-size effect), and the group input distributions differ in location (one group is intensive in each input). Under group-specific slopes no single log-linear frontier tightly envelopes all groups; the true envelope is the pointwise maximum $\max_j \mathbf{x}'_i \beta_j$, the true TGR $\exp(\mathbf{x}'_i \beta_g - \max_j \mathbf{x}'_i \beta_j)$ varies across observations within a group, and the estimated metafrontier coefficients converge to a pseudo-true tightest linear envelope that depends on the input distribution. Scenario B is therefore evaluated on the recovery of the efficiency quantities (TE, TE*, TGR) rather than on coefficient bias. Each scenario is replicated 500 times, and in each replication we estimate the deterministic SFA, stochastic SFA, and DEA metafrontiers. The complete simulation code for this section, including both scenario loops and the table construction, is provided in the supplementary replication script (Targets 3 and 7 to 10).

6.2 Parameter recovery

Table 5 reports the bias, Monte Carlo standard error (MC SE), standard deviation, and root mean squared error (RMSE) of the metafrontier coefficient estimates across the 500 replications.

Table 5: Monte Carlo results for metafrontier coefficient estimation (500 successful replications out of 500). MC SE is the Monte Carlo standard error of the mean estimate; RMSE is the root mean squared error.

Parameter	Method	True	Bias	MC SE	SD	RMSE
beta_1	Deterministic	0.5	0.0005	0.0006	0.0135	0.0135
beta_1	Stochastic	0.5	0.0008	0.0006	0.0145	0.0145
beta_2	Deterministic	0.3	0.0000	0.0006	0.0143	0.0143
beta_2	Stochastic	0.3	-0.0002	0.0006	0.0143	0.0143
Intercept	Deterministic	1.0	-0.0269	0.0042	0.0933	0.0971
Intercept	Stochastic	1.0	-0.1679	0.0052	0.1159	0.2040

All 500 replications complete without error and with optimiser convergence code zero at every stage, which is expected given the well-behaved Cobb–Douglas DGP with moderate noise levels; convergence failures are more likely with empirical data featuring collinear regressors or very small samples. In applied work, the `check_convergence()` function and the convergence block printed by `summary()` let users verify all estimation stages before interpreting TGRs, confidence intervals, or productivity decompositions. Figure 6 displays the distribution of coefficient bias across replications for both estimators.

Because the objective argument offers both identification criteria of Battese et al. (2004), we also refit the deterministic metafrontier under the squared-deviations QP objective on the first 50 Scenario A designs. The maximum absolute difference between LP- and QP-based TGRs is 2.5×10^{-12} ; on this design the binding-constraint pattern coincides and the two criteria are numerically indistinguishable, although they can diverge at extreme input mixes.

Both estimators recover the true slope coefficients (β_1, β_2) with negligible bias. The stochastic metafrontier intercept shows a persistent downward bias because the composed error $v^* - u^*$ has non-zero mean $-\sigma^* \sqrt{2/\pi}$ under a half-normal specification, so the second-stage SFA absorbs part of the intercept into the inefficiency term. This is a structural feature of the two-step stochastic metafrontier (Huang et al., 2014): slope coefficients remain consistent as $n \rightarrow \infty$, but the intercept does not. Applied users interested in level-unit intercept interpretation should add back $\hat{\sigma}^* \sqrt{2/\pi}$ or rely on the deterministic metafrontier, whose intercept is unbiased in our Monte Carlo.

6.3 Efficiency recovery

Table 6 reports, for each of the three estimators under Scenario A, the mean estimated technical efficiency against the own-group frontier (TE), the mean metafrontier efficiency (TE*), and the mean technology gap ratio (TGR), together with the corresponding true means and the Spearman rank correlations between estimated and true values.

Two features of Table 6 require care in interpretation. First, the DEA rows should be read as robustness of the framework under a noisy DGP: with $\sigma_v > 0$, observations with favourable noise draws define the DEA frontiers, so DEA efficiency levels are contaminated by construction and only their rankings are informative. Second, under the intercept-only Scenario A the true TGR takes only

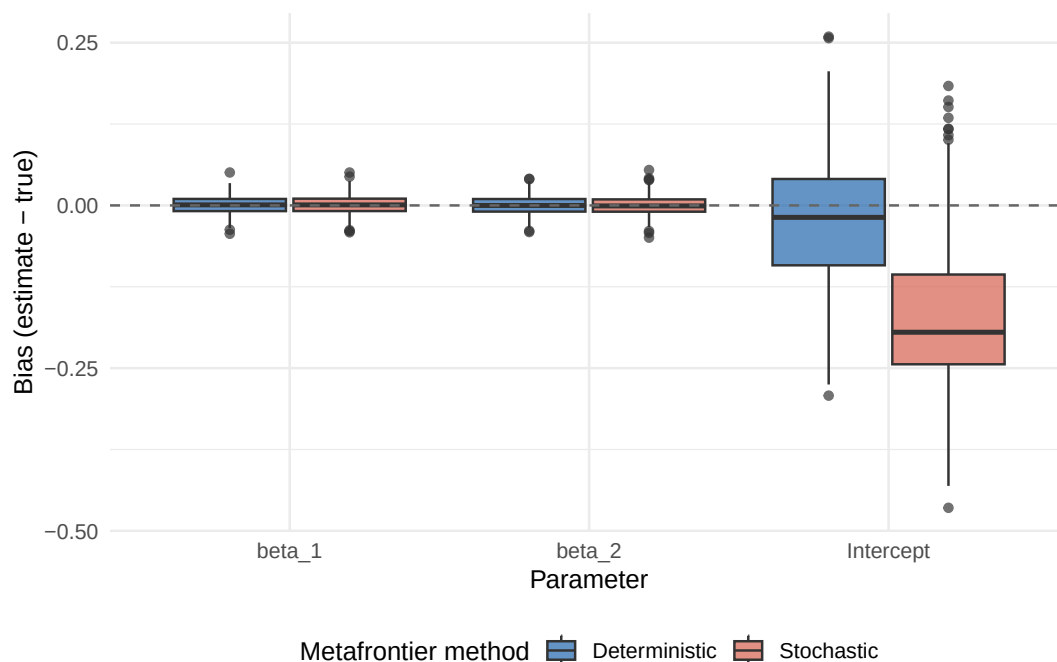


Figure 6: Distribution of metafrontier coefficient bias across 500 Monte Carlo replications for the deterministic and stochastic estimators (drawn with ggplot2; code in the supplementary materials). Boxes show the interquartile range and median; whiskers extend to 1.5 times the IQR.

Table 6: Efficiency recovery under Scenario A (means over 500 replications). TE is efficiency against the own-group frontier, TE* against the metafrontier, and TGR the technology gap ratio; Corr denotes the Spearman rank correlation between estimated and true values within a replication.

Estimator	TE (true)	TE (est)	Corr(TE)	TE* (true)	TE* (est)	Corr(TE*)	TGR (true)	TGR (est)	Corr(TGR)
Deterministic SFA	0.829	0.850	0.575	0.697	0.717	0.897	0.835	0.840	0.883
Stochastic SFA	0.829	0.850	0.575	0.697	0.827	0.880	0.835	0.969	0.866
DEA (VRS)	0.829	0.891	0.502	0.697	0.830	0.730	0.835	0.929	0.833

two distinct values (one per group), so the TGR rank correlation mostly reflects the between-group ordering; Scenario B below provides within-group TGR variation and hence a sharper test.

Table 7 reports the same statistics under Scenario B (three groups with group-specific slopes, correlated inputs, and group-specific input distributions), where the true TGR varies across observations within every group. Of the 500 replications, 500 complete successfully for all three estimators.

Table 7: Efficiency recovery under Scenario B: three groups with group-specific slopes, correlated log inputs, and group-specific input means (means over successful replications). The true TGR is computed against the pointwise maximum over the group frontiers; column definitions follow the Scenario A table.

Estimator	TE (true)	TE (est)	Corr(TE)	TE* (true)	TE* (est)	Corr(TE*)	TGR (true)	TGR (est)	Corr(TGR)
Deterministic SFA	0.828	0.845	0.547	0.766	0.513	0.644	0.925	0.608	0.761
Stochastic SFA	0.828	0.845	0.547	0.766	0.852	0.661	0.925	1.010	0.794
DEA (VRS)	0.828	0.898	0.502	0.766	0.810	0.566	0.925	0.902	0.731

Under Scenario B the estimated metafrontier coefficients converge to a pseudo-true tightest linear envelope rather than to a data-generating parameter vector, so no coefficient-bias table is reported. The efficiency rows in Table 7 show what this implies in practice. A single log-linear envelope cannot bend around three crossing group frontiers, so it lies above the pointwise-maximum envelope over much of the input space, and the deterministic TGR levels are understated by construction (mean 0.608 against a true mean of 0.925); the residual-based stochastic TGR centres near one regardless of the truth (mean 1.010). Rankings survive in both cases, with TGR rank correlations between 0.76 and 0.79. DEA imposes no functional form and recovers the TGR levels almost without bias (mean 0.902) at similar rank correlations. This contrast is the practical content of Scenario B: when group frontiers differ in slope and not merely in level, the DEA-based metafrontier is the safer choice for TGR levels, while the SFA-based variants still order firms and groups reliably.

6.4 Small-sample and unbalanced-group robustness

To assess performance under more challenging conditions, we repeat the Scenario A simulation with 30 firms per group (vs. 100 in the baseline), keeping all other DGP parameters unchanged.

With only 30 observations per group, 500 of 500 replications complete successfully (here and below, “successfully” means the estimation finished without error; non-zero optimiser codes are recorded separately and can be inspected with `check_convergence()`). The deterministic estimator shows slope biases of 0 (β_1) and -0.002 (β_2), modestly larger than the baseline. The TGR rank correlation remains high at 0.869. The framework ranks group technologies reliably even with small samples.

Empirical metafrontier applications frequently involve groups of markedly different sizes, and the group frontiers are estimated separately, so a small group’s frontier (and hence its TGR) can be unstable even when the total sample is large. To assess this, we simulate a strongly unbalanced design with 200 firms in the frontier-defining group and only 20 firms in the lagging group (a 10:1 ratio, with the technology gap on the small group), keeping all other Scenario A parameters unchanged.

Of the 500 replications, 500 complete without error and 500 additionally have optimiser convergence code zero at every stage. The deterministic estimator’s slope biases (0 for β_1 , -0.001 for β_2) remain modest, and the overall TGR rank correlation is 0.598. The instability concentrates where expected: the mean TGR bias in the 20-firm group is -0.019. The small group’s own frontier estimate drives this bias, since the pooled metafrontier is estimated from all 220 observations. The large group shows no comparable bias. Applied users with severely unbalanced groups should therefore interpret the small groups’ TGRs with caution and lean on the bootstrap intervals of `boot_tgr()`, which reflect the group-specific sample sizes.

7 Comparison with related packages

Table 8 provides a feature-level comparison of `metafrontier` with related R packages for efficiency analysis. All compared packages are on CRAN; `smfa` is the CRAN release (April 2026) of the GitHub package `metafrontierR` cited in the introduction. The table deliberately includes rows on which `metafrontier` offers nothing: the standard (single-technology) Malmquist index, the smoothed DEA score bootstrap of Simar and Wilson, and the sample selection metafrontier are all better served elsewhere.

Table 8: Feature comparison across R packages for efficiency and metafrontier analysis. N/A means not applicable (the package is a model engine, not a metafrontier framework); notes (a) and (b) are explained in the text below.

Feature	metafrontier	smfa	deaR	Benchmarking	frontier	sfaR	sfa
Deterministic metafrontier	LP + QP	LP + QP	No	No	No	No	No
Stochastic metafrontier	Yes	Yes	No	No	No	No	No
DEA-based metafrontier	Yes	No	Manual (a)	Manual (a)	No	No	No
Latent class metafrontier	Yes	Yes	No	No	No	No	No
Metafrontier Malmquist (O'Donnell et al. 2008)	Yes	No	No	No	No	No	No
Standard Malmquist index	No	No	Yes	Yes	No	No	No
Bootstrap TGR inference	Yes	No	No	No	No	No	No
DEA score bootstrap (Simar–Wilson)	No	No	Yes	Yes	No	No	No
Murphy–Topel correction	Yes	No	No	No	No	No	No
Panel SFA (time-varying)	Yes	No	No	No	Yes	No	Yes
Unbalanced panel SFA	Yes	No	No	No	Yes	No	Yes
Sample selection MF	No	Yes	No	No	No	No	No
Directional distance functions	Yes (b)	No	Yes	Yes	No	No	No
Hyperbolic (graph) efficiency	Yes	No	No	Yes	No	No	No
FDH technology	Yes	No	Yes	Yes	No	No	No
Slack analysis	Yes	No	Yes	Yes	No	No	No
ggplot2 visualisation	Yes	No	Yes	No	No	No	No
External model import	SFA fits	sfaR only	No	No	N/A	N/A	N/A
S3 model methods	38	10	Yes	Yes	Yes	Yes	Basic

Two cells of Table 8 carry notes. Note (a): pooled-reference scoring can be assembled manually in `deaR` and `Benchmarking` by scoring against a user-supplied reference set, but neither package forms, summarises, or provides inference for technology gap ratios. Note (b): custom direction vectors and firm-specific direction matrices are supported; `deaR` (`model_dir()`) and `Benchmarking` (`DIRECT`) also accept custom directions.

The `metafrontier` package is, to our knowledge, the only R package that combines SFA-based and DEA-based metafrontier analysis with the metafrontier Malmquist decomposition and TGR-specific inference in a single framework. Several complementary strengths of the other packages deserve acknowledgement. The `smfa` package (Owili, 2026) offers both deterministic criteria and two stochastic formulations, implements latent class group frontiers via `sfaR`’s `sfa1cmcross()`, and uniquely provides

the sample selection metafrontier of Greene (2010) for non-random group composition. The **deaR** package (Coll-Serrano et al., 2023) provides a broad DEA toolkit (including fuzzy DEA and the Simar–Wilson bootstrap) that extends well beyond metafrontier analysis. The **sfaR** (Dakpo et al., 2023) and **frontier** (Coelli and Henningsen, 2020) packages remain leading engines for stochastic frontier estimation, with **sfaR** in particular supporting ten inefficiency distributions against the three implemented here; the newer **sfa** package (Bernstein et al., 2026) provides a recent alternative with flexible panel specifications. The **metafrontier** package’s interoperability and engine layers are designed to complement rather than replace these estimators.

What does unification add beyond convenience? Two things. First, the three estimators return identical S3 objects, so cross-method robustness checks, as demonstrated in the real-data illustration of Section 2.4.6, require no glue code; discrepancies between a deterministic envelope, a stochastic second stage, and a nonparametric pooled frontier are themselves diagnostic. Second, and more fundamentally, the technology gap ratio is a *generated* quantity: it compares two estimated frontiers, so its sampling uncertainty is not available from any single-frontier package. The inference layer built around it (the TGR bootstrap of `boot_tgr()` and the Murphy–Topel correction for the stochastic metafrontier’s generated-regressor problem) exists, to our knowledge, in no other R package, and is the part of the workflow that cannot be assembled from the engines in Table 8.

8 Summary and future directions

The **metafrontier** package provides a unified R framework for metafrontier production function analysis. Its principal contribution is the integration of three metafrontier approaches (deterministic, stochastic, and DEA) with the metafrontier Malmquist decomposition, bootstrap inference, Murphy–Topel correction, and latent class estimation under a consistent S3 interface. All simulations complete in under ten minutes in the computational environment described in Section 2.3.4, and Section 2.4 demonstrates the package on real data from **sfaR**; additional empirical examples are provided in the package vignette (`vignette("metafrontier")`).

Three limitations should be noted. First, the SFA-based metafrontiers require the researcher to commit to a parametric functional form (for example Cobb–Douglas or translog) through the model formula, and the same form is imposed on every group frontier and on the metafrontier envelope; the package performs no specification search, so a misspecified form propagates from the group frontiers into the estimated technology gap ratios. The DEA-based estimators avoid this choice, at the cost of ignoring statistical noise. Second, the package estimates production frontiers only ($\varepsilon = v - u$); cost frontier support ($\varepsilon = v + u$) is planned. Third, in the SFA-based Malmquist decomposition (`method = "sfa"` in `malmquist_meta()`), the distance of each observation to the metafrontier is approximated by evaluating all estimated group frontiers at that observation’s inputs and taking the largest predicted output, rather than by re-estimating the enveloping metafrontier of Equation (4) in each period. This pointwise maximum coincides with the metafrontier of O’Donnell et al. (2008) wherever one group frontier dominates all others, but can understate it where group frontiers cross, so the resulting TGC and TC* components are approximate; the DEA-based Malmquist (the default) computes exact distances to the pooled frontier and should be preferred when an exact decomposition is required. The function documents this approximation and announces it at run time. Planned extensions include a sample selection metafrontier, spatial SFA metafrontiers, a Bayesian estimator propagating first-stage uncertainty, and multiplicatively complete, transitive (proper) TFP indexes such as the Färe–Primont and Lowe indexes (O’Donnell, 2012, 2018).

The package is available on CRAN (<https://CRAN.R-project.org/package=metafrontier>) and is developed on GitHub at <https://github.com/iik1/metafrontier>.

Acknowledgements and declaration of tool use

The author thanks the editor and three anonymous reviewers for comments that substantially improved both the package and this article. Large language model tools (Anthropic Claude) were used to assist with drafting portions of the manuscript text and package documentation and with code review. All methods, code, results, and the final wording were written and verified by the author and remain the author’s sole responsibility.

References

- D. Aigner, C. A. K. Lovell, and P. Schmidt. Formulation and estimation of stochastic frontier production function models. *Journal of Econometrics*, 6(1):21–37, 1977. doi: 10.1016/0304-4076(77)90052-5. [p418]

- G. E. Battese and T. J. Coelli. Prediction of firm-level technical efficiencies with a generalized frontier production function and panel data. *Journal of Econometrics*, 38(3):387–399, 1988. doi: 10.1016/0304-4076(88)90053-X. [p419]
- G. E. Battese and T. J. Coelli. Frontier production functions, technical efficiency and panel data: With application to paddy farmers in India. *Journal of Productivity Analysis*, 3(1–2):153–169, 1992. doi: 10.1007/BF00158774. [p431, 432]
- G. E. Battese and T. J. Coelli. A model for technical inefficiency effects in a stochastic frontier production function for panel data. *Empirical Economics*, 20(2):325–332, 1995. doi: 10.1007/BF01205442. [p431]
- G. E. Battese and D. S. P. Rao. Technology gap, efficiency, and a stochastic metafrontier function. *International Journal of Business and Economics*, 1(2):87–93, 2002. URL <https://ideas.repec.org/a/ijb/journal/v1y2002i2p87-93.html>. [p419]
- G. E. Battese, D. S. P. Rao, and C. J. O'Donnell. A metafrontier production function for estimation of technical efficiencies and technology gaps for firms operating under different technologies. *Journal of Productivity Analysis*, 21(1):91–103, 2004. doi: 10.1023/B:PROD.0000012454.06094.29. [p417, 418, 419, 424, 434]
- D. H. Bernstein, C. F. Parmeter, and A. Stead. *sfa: Stochastic Frontier Analysis*, 2026. URL <https://CRAN.R-project.org/package=sfa>. R package version 1.0.4. [p417, 437]
- P. Bogetoft and L. Otto. *Benchmarking with DEA, SFA, and R*. Springer, New York, 2011. doi: 10.1007/978-1-4419-7961-2. [p417]
- A. Charnes, W. W. Cooper, and E. Rhodes. Measuring the efficiency of decision making units. *European Journal of Operational Research*, 2(6):429–444, 1978. doi: 10.1016/0377-2217(78)90138-8. [p420]
- T. Coelli and A. Henningsen. *frontier: Stochastic Frontier Analysis*, 2020. URL <https://CRAN.R-project.org/package=frontier>. R package version 1.1-8. [p417, 437]
- V. Coll-Serrano, V. Bolós, and R. Benítez Suárez. *deaR: Conventional and Fuzzy Data Envelopment Analysis*, 2023. URL <https://CRAN.R-project.org/package=deaR>. R package version 1.5.4. [p418, 437]
- K. H. Dakpo, Y. Desjeux, and L. Latruffe. *sfaR: Stochastic Frontier Analysis Using R*, 2023. URL <https://CRAN.R-project.org/package=sfaR>. R package version 1.0.0. [p417, 437]
- W. H. Greene. A stochastic frontier model with correction for sample selection. *Journal of Productivity Analysis*, 34(1):15–24, 2010. doi: 10.1007/s11123-009-0159-1. [p418, 437]
- C. J. Huang, T.-H. Huang, and N.-H. Liu. A new approach to estimating the metafrontier production function based on a stochastic frontier framework. *Journal of Productivity Analysis*, 42(3):241–254, 2014. doi: 10.1007/s11123-014-0402-2. [p417, 419, 425, 429, 434]
- J. Jondrow, C. A. K. Lovell, I. S. Materov, and P. Schmidt. On the estimation of technical inefficiency in the stochastic frontier production function model. *Journal of Econometrics*, 19(2–3):233–238, 1982. doi: 10.1016/0304-4076(82)90004-5. [p419]
- A. Kneip, L. Simar, and P. W. Wilson. Testing hypotheses in nonparametric models of production. *Journal of Business & Economic Statistics*, 34(3):435–456, 2016. doi: 10.1080/07350015.2015.1049747. [p426]
- K. Konis and F. Schwendinger. *lpSolveAPI: R Interface to lp_solve*, 2024. URL <https://CRAN.R-project.org/package=lpSolveAPI>. R package version 5.5.2.0-17.12. [p419, 423]
- S. C. Kumbhakar and C. A. K. Lovell. *Stochastic Frontier Analysis*. Cambridge University Press, Cambridge, 2000. doi: 10.1017/CBO9781139174411. [p419]
- V. H. Moreira and B. E. Bravo-Ureta. Technical efficiency and metatechnology ratios for dairy farms in three southern cone countries: A stochastic meta-frontier model. *Journal of Productivity Analysis*, 33(1):33–45, 2010. doi: 10.1007/s11123-009-0144-8. [p417]
- K. M. Murphy and R. H. Topel. Estimation and inference in two-step econometric models. *Journal of Business & Economic Statistics*, 3(4):370–379, 1985. doi: 10.1080/07350015.1985.10509471. [p418, 420, 430]
- C. J. O'Donnell. An aggregate quantity framework for measuring and decomposing productivity change. *Journal of Productivity Analysis*, 38(3):255–272, 2012. doi: 10.1007/s11123-012-0275-1. [p421, 437]

- C. J. O'Donnell. *Productivity and Efficiency Analysis: An Economic Approach to Measuring and Explaining Managerial Performance*. Springer, Singapore, 2018. doi: 10.1007/978-981-13-2984-5. [p421, 437]
- C. J. O'Donnell, D. S. P. Rao, and G. E. Battese. Metafrontier frameworks for the study of firm-level efficiencies and technology ratios. *Empirical Economics*, 34(2):231–255, 2008. doi: 10.1007/s00181-007-0119-4. [p417, 418, 419, 420, 421, 437]
- S. Olieko. *metafrontierR: Stochastic and Deterministic Metafrontier Analysis*, 2026. URL <https://github.com/SulmanOlieko/metafrontierR>. R package version 1.0.0. [p418]
- S. O. Owili. *smfa: Stochastic Metafrontier Analysis*, 2026. URL <https://CRAN.R-project.org/package=smfa>. R package version 1.0.0. [p418, 436]
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL <https://www.R-project.org/>. [p417]
- L. Simar and V. Zelenyuk. Statistical inference for aggregates of Farrell-type efficiencies. *Journal of Applied Econometrics*, 22(7):1367–1394, 2007. doi: 10.1002/jae.991. [p426]
- B. A. Turlach and A. Weingessel. *quadprog: Functions to Solve Quadratic Programming Problems*, 2019. URL <https://CRAN.R-project.org/package=quadprog>. R package version 1.5-8. [p419, 423]
- H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag, New York, 2016. doi: 10.1007/978-3-319-24277-4. [p427]

Erik Enstad
NHH Norwegian School of Economics
Department of Business and Management Science
Bergen, Norway
ORCID: 0009-0006-1053-3849
erik.enstad@nhh.no